FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

2

RULE – BASED AGENT

Student Name	Eric V. Tividad Jr.
Section:	AN 42
Professor:	Ms. Crisola Tan
Date:	10/2/2026

I. BACKGROUND INFORMATION

A Rule-Based Agent is a type of intelligent agent that operates by following predefined rules to decide its actions. In this lab, students will work with a vacuum cleaner agent that can clean rooms or move between rooms depending on their state (dirty or clean).

The goal is to understand how agents perceive the environment, act according to rules, and update the environment.

II. TECHNICAL ASSESSMENT

RULE – BASED AGENT

Objective

At the end of this laboratory activity, students are expected to:

1. Explain the concept of rule-based agents.
2. Implement a rule-based agent in Python.
3. Interact with an environment by perceiving and acting based on rules.
4. Observe and analyze the behavior of an agent in a controlled environment.

Materials Needed

- Computer with Python 3.x installed
- Text editor or IDE (e.g., VSCode, PyCharm)
- Optional: Jupyter Notebook for interactive testing

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA2_lastname.pdf**
   (Example: TA2_Patilano.pdf)
8. Submit the **python file** and **PDF** file.

Laboratory Task

Pre-Lab Preparation

1. Review the Python concepts:
   - Classes and objects
   - If-Else statements

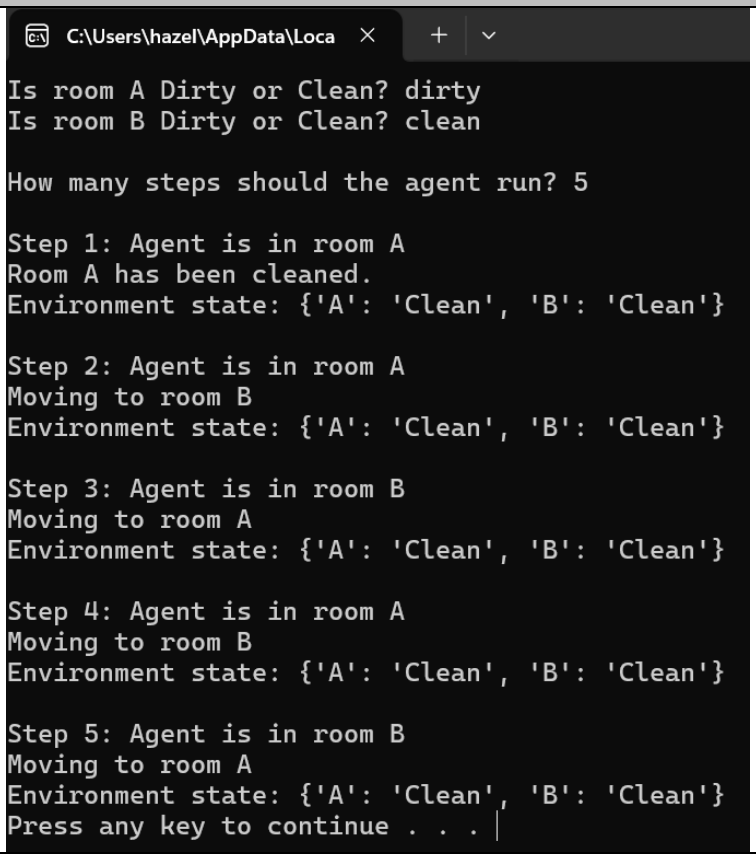
- Loops (for loop)
- Functions / methods

2. Understand the environment:
   - Two rooms: A and B
   - Each room can be Dirty or Clean

Task 1: Set Up the Environment

1. Initialize two rooms using a Python dictionary.
2. Allow the user to set the **initial state** of each room (dirty or clean).
3. Write a function to **check if a room is dirty** and a function to **clean a room**.

Sample Output



Task 2: Implement the Rule-Based Agent

1. Create a class for the agent.
2. Implement the **perceive-and-act method**:
   - Rule 1: If the current room is dirty, clean it.
   - Rule 2: If the current room is clean, move to the other room.
3. Implement a **move method** to switch rooms.

Task 3: Run the Simulation

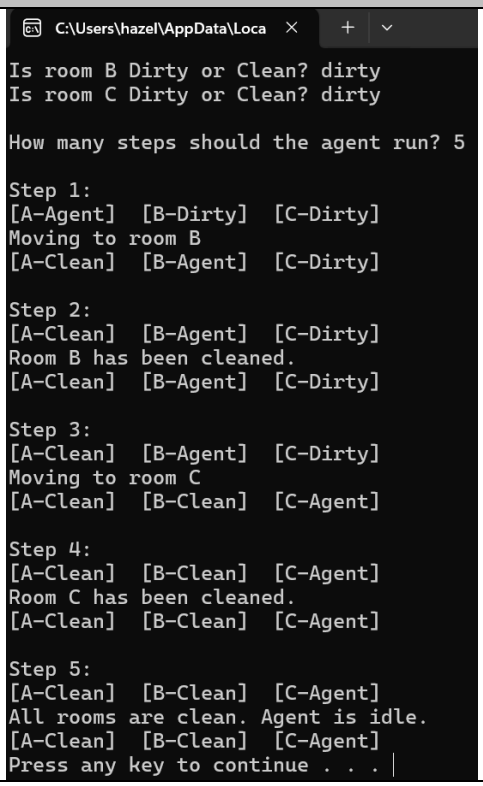
1. Ask the user how many steps the simulation should run.
2. Run the agent in a loop for the specified steps.

3. After each step, print:
    - The agent's current location
    - The action taken (cleaned/moved)
    - Current state of the environment

Optional (Bonus Task)

- Add a **third room (C)** and update the agent logic.
- Make the agent **randomly choose which dirty room to clean next**.
- Visualize the agent moving between rooms using a **text-based grid**.

Sample Output (Bonus Task)



Your Code - Laboratory Tasks ⇩

```
import random

# task 1
# two rooms: A and B
rooms = {"A": "Clean", "B": "Clean"}

# checks if a room is dirty
def is_dirty(room):
    if rooms[room] == "Dirty":
```

```
        return True
    return False

# cleans a room
def clean(room):
    rooms[room] = "Clean"
    print("I am cleaning the room")

# task 2
class Agent:
    def __init__(self):
        self.location = "A"   # agent always starts in room A

    # moves to another room
    def move(self):
        if self.location == "A":
            self.location = "B"
        else:
            self.location = "A"

    def perceive_and_act(self):
        # room dirty = clean it
        if is_dirty(self.location):
            clean(self.location)
            print("Room " + self.location + " has been cleaned.")

        # room is clean = move to other
        else:
            self.move()
            print("Moving to room " + self.location)

# for running basic simulation
def run_basic():
    # ask the user for the starting state of each room
    for r in rooms:
        state = input("Is room " + r + " Dirty or Clean? ")
        rooms[r] = state.strip().capitalize()

    steps = int(input("\nHow many steps should the agent run? "))

    agent = Agent()

    for i in range(1, steps + 1):
        print("\nStep " + str(i) + ": Agent is in room " + agent.location)
```

```
        agent.perceive_and_act()
        print("Environment state:", rooms)

# bonus task section
rooms3 = {"A": "Clean", "B": "Clean", "C": "Clean"}

# new agent class for the bonus simulation
class Agent3:
    def __init__(self):
        self.location = "A"   # starts in room A again

    # same logic on the first agent; just moves randomly on the three rooms to clean it
    def perceive_and_act(self):
        # room is dirty = clean it
        if rooms3[self.location] == "Dirty":
            rooms3[self.location] = "Clean"
            print("Room " + self.location + " has been cleaned.")
        else:
            # perceive and collect dirty rooms for random picking
            dirty_rooms = []
            for r in rooms3:
                if rooms3[r] == "Dirty":
                    dirty_rooms.append(r)

            if len(dirty_rooms) > 0:
                self.location = random.choice(dirty_rooms)
                print("Moving to room " + self.location)
            else:
                # all rooms are cleaned!
                print("All rooms are clean. Agent is idle")

 # prints the rooms like [A-Agent]  [B-Dirty]  [C-Clean]
def show_grid(location):
    line = ""
    for r in rooms3:
        if r == location:
            line += "[" + r + "-Agent]  "
        else:
            line += "[" + r + "-" + rooms3[r] + "]  "
    print(line)

 # runs bonus simulation
def run_bonus():
    for r in rooms3:
```

```
        state = input("Is room " + r + " Dirty or Clean? ")
        rooms3[r] = state.strip().capitalize()

    steps = int(input("\nHow many steps should the agent run? "))

    agent = Agent3()

    for i in range(1, steps + 1):
        print("\nStep " + str(i) + ":")
        show_grid(agent.location)
        agent.perceive_and_act()
        show_grid(agent.location)

# ze menu
print("Technical Assesment 2: Rule-Based Agent")
print("1 - Two rooms")
print("2 - Three rooms (Bonus)")
choice = input("Pick a mode: ")
print()

if choice == "1":
    run_basic()
else:
    run_bonus()
```

Output – Basic Simulation:

```
PS C:\Users\ej\Desktop\1School\4th Yr - 1st Sem\Intelligent Systems\Technical Assesments> & C:
School/4th Yr - 1st Sem/Intelligent Systems/Technical Assesments/ta2.py"
Technical Assesment 2: Rule-Based Agent
1 - Two rooms
2 - Three rooms (Bonus)
Pick a mode: 1

Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? clean

How many steps should the agent run? 3

Step 1: Agent is in room A
I am cleaning the room
Room A has been cleaned.
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 2: Agent is in room A
Moving to room B
Environment state: {'A': 'Clean', 'B': 'Clean'}

Step 3: Agent is in room B
Moving to room A
Environment state: {'A': 'Clean', 'B': 'Clean'}
PS C:\Users\ej\Desktop\1School\4th Yr - 1st Sem\Intelligent Systems\Technical Assesments>
```

Output – Bonus Simulation:

```
Technical Assesment 2: Rule-Based Agent
1 - Two rooms
2 - Three rooms (Bonus)
Pick a mode: 2

Is room A Dirty or Clean? dirty
Is room B Dirty or Clean? dirty
Is room C Dirty or Clean? dirty

How many steps should the agent run? 5

Step 1:
[A-Agent]  [B-Dirty]  [C-Dirty]
Room A has been cleaned.
[A-Agent]  [B-Dirty]  [C-Dirty]

Step 2:
[A-Agent]  [B-Dirty]  [C-Dirty]
Moving to room B
[A-Clean]  [B-Agent]  [C-Dirty]

Step 3:
[A-Clean]  [B-Agent]  [C-Dirty]
Room B has been cleaned.
[A-Clean]  [B-Agent]  [C-Dirty]

Step 4:
[A-Clean]  [B-Agent]  [C-Dirty]
Moving to room C
[A-Clean]  [B-Clean]  [C-Agent]

Step 5:
[A-Clean]  [B-Clean]  [C-Agent]
Room C has been cleaned.
[A-Clean]  [B-Clean]  [C-Agent]
```

Answer the Questions ⇩

Task 4: Analysis Questions

After running the simulation, answer the following:

1. What rules does the agent follow?
    - I made two agents that technically follows the same rule, but the other one has randomization added into it. First agent does clean a room if it considered dirty, and moves to the other room if the room is considered clean. On the other hand, the second agent cleans a dirty room, but also collects dirty room if the first room is clean. This for the random picking of rooms to clean.

2. How does the agent decide when to clean and when to move?
    - It can only clean a room if it has the state is set to dirty, and moving to other rooms is only done once a room is considered clean or the agent is still in the same room.

3. What happens if both rooms are already clean at the start?
    - The program will still run due to the step inputted. Either the agent will move to other rooms or just stay idle, depending on the inputted steps.

4. How would you modify the agent to handle **more than two rooms**?
    - I think I just would change the move function of the agent. Instead of explicitly moving to room A to B, I would change it where it would move to the next room instead of explicitly moving to room B instead.